

Local AI Software Team

Product Idea Document

1. Product Concept

Local AI Software Team is a local-first product that acts like a complete software team. The user gives a rough requirement, and the system helps transform it into a complete software delivery package.

The product acts as:

- Business Analyst
- Product Owner
- Project Manager
- Solution Architect
- Developer
- QA Engineer
- Code Reviewer
- Security Reviewer
- DevOps/Release Assistant

The goal is not only to write code. The goal is to manage the full journey from rough idea to deliverable software.

2. One-line Pitch

A local AI software team that turns rough requirements into clear specs, tasks, code, tests, reviews, and delivery reports using the user's existing AI coding tools.

3. Core Vision

Most AI coding tools are good when the user already knows exactly what to build.

But real software work usually starts like this:

- "I need an approval flow."
- "I want users to export invoices."
- "I need a dashboard for admins."
- "I want a booking cancellation feature."

These inputs are vague. They lack business rules, edge cases, acceptance criteria, technical scope, test cases, and delivery plan.

Local AI Software Team solves this by acting like a real software team that receives the vague input, asks questions, makes assumptions, creates documents, breaks work into tasks, coordinates AI coding tools, verifies the result, and produces a delivery report.

4. Product Positioning

This product is not a Devin clone.

It should be positioned as:

The software delivery layer above AI coding agents.

Or:

An AI BA, PM, Architect, QA, and Reviewer for your existing coding agents.

The product does not try to replace Claude Code, Codex, Gemini CLI, Aider, Cursor, or other coding agents.

Instead, it organizes them into a structured software team workflow.

5. Target Users

5.1 Primary Users

- Solo developers
- Indie hackers
- Startup founders
- Small software teams
- Freelancers
- Outsourcing teams
- Backend developers
- Technical leads
- Developers who often receive vague requirements

5.2 Initial Niche

The best first niche:

Vietnamese requirement → clear software delivery package for Java/Spring Boot backend projects.

Why this niche is strong:

- Many Vietnamese developers receive requirements in Vietnamese.
- Requirements are often unclear or incomplete.
- Developers spend a lot of time converting business language into technical tasks.
- Java/Spring Boot is common in enterprise backend systems.
- Enterprise features often require docs, tests, approval flows, reports, and traceability.

6. Main Problem

AI coding tools can write code, but they do not fully solve the software delivery process.

Common problems:

1. Requirement is vague.
2. Business rules are missing.
3. Edge cases are not defined.
4. Acceptance criteria are unclear.
5. No test plan exists.
6. No technical design exists.
7. No task breakdown exists.
8. AI may code before understanding the requirement.
9. AI may create code that does not match business intent.
10. There is no traceability from requirement to code and tests.

The real pain is not only coding.

The real pain is:

Turning messy business intent into structured software work.

7. Product Solution

Local AI Software Team takes a rough input and turns it into a structured workflow.

Example input:

“Add booking cancellation approval. If refund amount is over 50 million VND, director approval is required. Accounting also needs to process refund and VinClub points.”

The product creates:

- clarified requirement
- business rules
- open questions
- assumptions
- acceptance criteria
- product scope
- technical design
- task breakdown
- test matrix
- implementation plan
- coding agent instructions
- code changes
- test result
- review report
- final delivery report
- traceability matrix

8. Key Product Promise

The product promise should be realistic.

8.1 Early Promise

Give the product a rough requirement. It will create clear requirements, docs, task breakdown, test cases, and implementation instructions.

8.2 Later Promise

Give the product a rough requirement and a project. It will coordinate AI agents to implement, test, review, and prepare a delivery report.

8.3 Long-term Promise

Manage an AI software team that can take a rough requirement and move it toward working software with minimal human management.

9. User Experience Concept

The user should feel like they are working with a team.

Instead of one chat box, the product should show different AI roles:

- BA Agent asks clarification questions.
- PM Agent breaks work into tasks.
- Architect Agent explains the design.
- Developer Agent works on implementation.
- QA Agent creates and checks test cases.
- Reviewer Agent checks the result.
- Reporter Agent summarizes the delivery.

The user can approve, reject, revise, pause, or continue at each stage.

10. Main Workflow

10.1 Requirement Intake

User enters a rough requirement.

Example:

“Build an expense approval system. Employee submits request, manager approves, finance pays, admin views report.”

The product identifies:

- goal
- users
- missing rules
- possible features

- possible risks
- unclear parts

10.2 Requirement Clarification

The AI BA creates:

- clarified requirement
- business rule table
- user roles
- main flow
- alternative flows
- edge cases
- assumptions
- open questions

If the requirement is too vague, the product asks questions.

If the user wants faster progress, the product can make assumptions and mark them clearly.

10.3 Product Scope

The AI Product Owner defines:

- MVP scope
- out-of-scope items
- priority
- success criteria
- release goal

This prevents the product from becoming too large.

10.4 Task Breakdown

The AI PM creates:

- epic
- user stories
- subtasks
- priority
- dependency
- definition of done

The output should be ready to copy into Jira, Linear, Trello, or any task management tool.

10.5 Design Planning

The AI Architect creates:

- high-level design

- affected modules
- API design
- database design
- service flow
- validation rules
- permission rules
- risks

10.6 Implementation

The AI Developer uses the selected coding agent to implement the approved plan.

The user should approve before implementation starts.

10.7 Testing

The AI QA creates:

- test scenarios
- unit test ideas
- integration test ideas
- manual test checklist
- acceptance criteria mapping

The product checks whether the implementation satisfies the requirement.

10.8 Review

The AI Reviewer checks:

- requirement coverage
- code quality
- maintainability
- security risks
- missing tests
- unrelated changes
- edge cases

10.9 Final Report

The product generates:

- summary of what was built
- files changed
- tests run
- test result
- remaining risks
- open questions
- next steps
- traceability matrix

11. Product Modes

11.1 Docs-only Mode

The product only creates documents and plans.

Best for:

- early MVP
- requirement analysis
- teams that do not want AI to edit code yet

Output:

- requirement doc
- acceptance criteria
- technical design
- task breakdown
- test matrix

11.2 Assisted Mode

The product creates prompts and instructions for coding agents.

The user manually runs the coding tool.

Best for:

- safer workflow
- users who already use Claude Code, Codex, Gemini CLI, or Aider

11.3 Semi-autonomous Mode

The product coordinates one coding agent to implement the approved plan.

Best for:

- solo developers
- small features
- controlled code changes

11.4 Team Mode

The product behaves like a full AI software team.

It can:

- clarify requirements
- create tasks

- coordinate agents
- run implementation
- review result
- generate report
- update external tools if enabled

12. Key Features

12.1 Requirement Clarifier

Turns vague input into clear requirement.

Outputs:

- clarified requirement
- user roles
- business rules
- assumptions
- open questions
- acceptance criteria

12.2 AI Team Room

A workspace where each AI role reports progress.

Example:

BA Agent:

I found 5 unclear business rules. Please confirm approval levels, refund behavior, rejection flow, notification recipients, and timeout behavior.

Architect Agent:

Proposed design is ready. The feature affects cancellation flow, approval policy, refund calculation, and notification logic.

QA Agent:

I created 18 test cases. 12 are normal cases, 4 are edge cases, and 2 are permission cases.

12.3 Task Generator

Creates tasks that can be used in Jira or similar tools.

Output:

- epic
- stories

- subtasks
- acceptance criteria
- definition of done

12.4 Documentation Generator

Creates:

- PRD
- SRS
- technical design
- API design
- database design
- test plan
- release note
- final report

12.5 Coding Agent Coordinator

Coordinates existing coding agents instead of replacing them.

The user can choose which AI coding tool to use.

12.6 Test Matrix Generator

Maps requirement to test cases.

Example:

Requirement	Test Case	Expected Result
Refund above 50 million requires director approval	Create refund request with 60 million VND	Director approval step is added
Refund below 50 million does not require director approval	Create refund request with 30 million VND	Director approval step is not added

12.7 Review Assistant

Reviews the result before the user accepts it.

Checks:

- Does this match the requirement?
- Are all acceptance criteria covered?
- Are there missing tests?
- Are there security risks?
- Are there unrelated changes?
- Is the solution too complex?

12.8 Traceability Matrix

This is a key differentiator.

It maps:

- requirement
- acceptance criteria
- task
- code files
- test cases
- test result

Example:

Requirement	Acceptance Criteria	Task	Code	Test	Status
Refund above 50 million requires director approval	AC-002	TASK-003	ApprovalPolicyService	shouldRequireDirectorApproval	Covered

13. External Tool Integrations

Integrations are optional.

The product should work without them.

13.1 GitHub / GitLab

Purpose:

- connect code changes to pull requests
- read review comments
- read build result
- create final summary

13.2 Jira

Purpose:

- create epics
- create stories
- create subtasks
- update task status
- add progress comments

- link delivery report

13.3 Slack / Microsoft Teams

Purpose:

- post progress updates
- ask clarification questions
- notify when review is ready
- notify final result

13.4 Confluence / Notion

Purpose:

- publish requirement docs
- publish technical design
- publish test plan
- publish final report

13.5 CI/CD Tools

Purpose:

- verify build
- verify tests
- detect failure
- ask AI agent to fix issues

14. Local-first Philosophy

The product should run locally first.

Why:

- user code stays private
- no backend required
- no user login required
- no cloud storage required
- easier to trust
- easier to adopt
- user can bring their own AI tools
- user can bring their own integration tokens

This makes the product attractive to developers and teams that are cautious about uploading private code to another SaaS platform.

15. User Control and Safety

The product should not blindly automate everything.

Important controls:

- user approval before coding
- user approval before external updates
- user approval before final delivery
- clear assumptions
- clear open questions
- clear risks
- ability to pause
- ability to reject plan
- ability to regenerate docs
- ability to review before accepting result

The product should behave like a reliable assistant team, not an uncontrolled autonomous system.

16. MVP Scope

The first MVP should not try to fully complete all projects automatically.

The first MVP should focus on:

Rough requirement → clear requirement → docs → task breakdown → test matrix → implementation instructions.

MVP features:

- local web interface
- requirement input
- AI role-based workflow
- requirement clarification
- acceptance criteria generation
- technical design generation
- task breakdown generation
- test matrix generation
- final report generation
- prompt templates for coding agents
- run history
- settings page

MVP should avoid:

- full autonomous coding
- complex Jira automation
- Slack bot event handling
- cloud backend
- billing
- enterprise admin
- automatic deployment

17. MVP Output Example

Input:

“Thêm API export invoice CSV theo hotelId, date range, status.”

Output:

1. Clarified requirement
2. Business rules
3. Acceptance criteria
4. Open questions
5. API design
6. Data filtering logic
7. Permission rules
8. Task breakdown
9. Test matrix
10. Implementation prompt
11. Final report

18. Product Roadmap

Phase 1: Requirement-to-Docs

Goal:

Make vague requirements clear.

Features:

- requirement clarification
- acceptance criteria
- business rules
- assumptions
- open questions
- docs export

Phase 2: Requirement-to-Tasks

Goal:

Turn clear requirements into software work.

Features:

- epic/story/subtask generation
- definition of done
- priority
- dependency

- test matrix

Phase 3: Local Coding Agent Coordination

Goal:

Use existing AI coding tools to implement approved plans.

Features:

- implementation instructions
- coding agent coordination
- changed files summary
- review report

Phase 4: Test and Review Workflow

Goal:

Make AI-generated work more reliable.

Features:

- test result summary
- QA review
- code review
- requirement coverage
- traceability matrix

Phase 5: External Tool Workflow

Goal:

Make the product feel like a real software team.

Features:

- Jira task creation
- Slack progress updates
- GitHub pull request summary
- documentation publishing

Phase 6: Team and Enterprise Readiness

Goal:

Support real software teams.

Features:

- shared templates
- approval policies
- audit logs
- team workflows
- enterprise security options

19. Business Model

19.1 Free Plan

For individual developers.

Features:

- local usage
- requirement clarification
- basic docs
- basic task breakdown
- basic test matrix

19.2 Pro Plan

For serious developers and freelancers.

Features:

- advanced docs
- advanced templates
- coding agent coordination
- review reports
- traceability matrix
- more run history

19.3 Team Plan

For small teams.

Features:

- Jira integration
- Slack integration
- GitHub integration
- shared templates
- team workflow
- approval policy
- audit report

19.4 Enterprise Plan

For companies.

Features:

- private deployment option
- stricter security
- team policies
- audit logs
- internal templates
- enterprise support

20. Differentiation

The product is different from simple AI coding tools because it focuses on the full software delivery process.

Most coding agents answer:

“How do I implement this?”

This product answers:

“What exactly should be built, how should it be planned, how should it be tested, and how do we know it is done?”

Key differentiation:

1. Requirement-first
2. Local-first
3. Multi-role AI team
4. Works above existing coding agents
5. Strong documentation
6. Strong task breakdown
7. Strong test planning
8. Strong traceability
9. Human approval workflow
10. Vietnamese requirement support

21. Example Scenario

User input:

“Tạo hệ thống approval cho nghỉ phép. Nhân viên tạo đơn, manager duyệt, HR xác nhận. Nếu nghỉ trên 5 ngày thì cần director duyệt.”

AI BA output:

- Employee can create leave request.
- Manager approves or rejects.
- HR confirms approved leave.
- Director approval is required when leave duration is more than 5 days.
- Rejection must include reason.
- Employee can see request status.

AI PM output:

Epic:

Leave Approval System

Stories:

1. Submit leave request
2. Manager approval
3. Director approval for long leave
4. HR confirmation
5. Status tracking
6. Test leave approval workflow

AI QA output:

Test cases:

1. Employee submits valid leave request.
2. Manager approves short leave.
3. Long leave requires director approval.
4. Rejected leave requires reason.
5. HR confirms approved leave.
6. Employee can see status history.

Final output:

- requirement doc
- technical design
- task breakdown
- test matrix
- implementation plan
- final report

22. Product Name Ideas

Possible names:

1. Local AI Team
2. TeamForge AI
3. SpecForge

4. ReqPilot
5. DeliveryMind
6. DevTeam AI
7. AgentPM
8. ShipForge
9. WorkForge
10. CodeTeam AI

Best options:

- SpecForge: good for requirement-to-spec
- TeamForge AI: good for AI software team positioning
- DeliveryMind: good for end-to-end delivery
- ReqPilot: simple and requirement-focused

23. Final Product Statement

Local AI Software Team is a local-first AI software team that helps developers turn rough requirements into clear software delivery outputs.

It acts like a BA, PM, Architect, Developer, QA, Reviewer, and Reporter.

It does not replace existing coding agents. It organizes them.

It does not require a cloud backend. It runs locally.

Its main value is not just writing code.

Its main value is turning vague business intent into structured, reviewable, testable, and traceable software delivery.

24. Short Final Pitch

A local AI software team that turns rough requirements into specs, tasks, code, tests, reviews, and delivery reports using your existing AI coding tools.

25. Long Final Pitch

Most AI coding tools are useful when the developer already knows what to build. But real software projects often begin with vague requirements, missing business rules, unclear acceptance criteria, and no test plan.

Local AI Software Team solves this by acting like a complete software team on the user's machine. It clarifies requirements, creates documentation, breaks work into tasks, prepares test plans, coordinates AI coding agents, reviews the result, and generates traceability reports.

It is not another coding agent.

It is the local delivery layer above coding agents.